

Part 1: Rebuild the forgotten basics

What is a Bitcoin transaction?

A Bitcoin transaction is a record that says:

- which old coins are being spent
- who is allowed to spend them
- where the new coins will go

Bitcoin does **not** store “Alice has 5 BTC” in one account table like a bank. Instead, it stores many small spendable pieces called **UTXOs**. A transaction takes some old UTXOs as input and creates new outputs as new UTXOs.

What is a UTXO?

UTXO = **Unspent Transaction Output**

Easy meaning:

a piece of Bitcoin that was created earlier and has not been spent yet.

So if somebody paid you before, that earlier payment created an output for you. If you still have not used it, that output is your UTXO.

Think of it like cash notes in your wallet.

You do not have “one giant balance object.”

You have separate pieces of money.

What is an input?

An **input** points to an older output and says:

“I want to spend that old coin.”

So input = reference to an older UTXO.

What is an output?

An **output** is the new coin being created in the new transaction.

Each output usually contains:

- a **value**
- a **locking condition**

That locking condition is usually written in **scriptPubKey**.

What is scriptPubKey?

This is one of the most important terms.

scriptPubKey = locking script

Easy meaning:

It is the condition that says who can spend this output later.

It is like putting money inside a digital lockbox and writing the rule on the box:

“Only the person with the correct proof can open this later.”

So when a transaction creates an output, it is not just sending money. It is locking that money with rules.

What is scriptSig?

scriptSig = unlocking data in legacy transactions.

Easy meaning:

This is the stuff the spender provides later to prove they are allowed to open the old lock.

Usually it contains:

- digital signature
- public key

So:

- old output has **scriptPubKey** = lock
- new input has **scriptSig** = key/proof to open that lock

Very important line:

Bitcoin checks whether the unlocking data satisfies the locking rule.

That is what your lecture means by:

If ScriptPubKey(ScriptSig), then create output

Easy meaning of that line:

If the old coin is successfully unlocked, then the transaction is valid and new outputs may be created.

What is P2PKH?

P2PKH = **Pay to Public Key Hash**

This is a very common old-style Bitcoin output.

It means the output is locked to the **hash of someone's public key**.

Later, the spender must provide:

- their public key
- their signature

and Bitcoin checks both.

Why hash the public key instead of storing full key directly?

Because public keys are big. A hash is shorter.

So instead of locking directly to the full public key, Bitcoin often locks to its hash. That saves space.

Part 2: The standard P2PKH story

Let's tell the normal story in simple words.

Suppose an earlier transaction created an output for Alia. That old output contains a scriptPubKey like:

- OP_DUP
- OP_HASH160
- pubKeyHash
- OP_EQUALVERIFY
- OP_CHECKSIG

You do not need to panic at the opcodes. The idea is simple.

That script is basically saying:

1. Show me your public key
2. I will hash it
3. It must match the stored public key hash
4. Then I will verify your signature
5. If all good, you may spend this coin

That is all.

So in plain English, P2PKH asks:

- Is this really the right public key?
- Did the real owner sign the transaction?

If yes, spend is valid.

Part 3: Multiple inputs and outputs

Your lecture shows that one transaction can have multiple inputs and multiple outputs.

Why multiple inputs?

Because one UTXO may not be enough.

Example:

You want to pay 0.5 BTC, but you only have:

- 0.2 BTC UTXO
- 0.3 BTC UTXO

So the transaction uses both inputs together.

It is like paying with two smaller notes.

Why multiple outputs?

Because one payment often produces more than one result.

Example:

You spend 1 BTC to pay someone 0.4 BTC.

Then transaction may create:

- 0.4 BTC to receiver
- 0.59 BTC back to you as change
- remaining tiny amount as fee

So Bitcoin commonly has:

- one output to payee
- one change output back to sender

This is super important for quizzes.

Part 4: Unspendable output and OP_RETURN

Your lecture also shows **OP_RETURN**.

What is OP_RETURN?

It creates an output that is intentionally **unspendable**.

Meaning: nobody can spend it later.

Why make such an output?

Usually to store small data on-chain, not to transfer money.

So this output is more like:

“record this information”

instead of

“give this Bitcoin to someone.”

Why usually zero BTC?

Because if the output can never be spent, you do not want to trap valuable BTC in it.

So the big takeaway:

Not every Bitcoin output is meant to become future spendable money. Some are just for data and are intentionally unspendable.

Part 5: ECDSA signature, r and s, DER encoding

Now we get to the scary-looking signature format part from Lecture 11.

What is ECDSA?

ECDSA = **E**lliptic **C**urve **D**igital **S**ignature **A**lgorithm

Easy meaning:

It is the signature system Bitcoin used for legacy signatures.

You do **not** need the curve math here.

For your quiz, think:

“ECDSA is the cryptographic method used to create digital signatures in legacy Bitcoin.”

What are r and s?

An ECDSA signature is basically made of two numbers:

- r
- s

Together, these two numbers form the signature.

Why is there a weird byte format around them?

Because Bitcoin does not store “r and s” as plain words.

It stores them in a structured byte format so computers can parse them properly.

Your lecture shows format like:

30 02 02

Do not memorize every symbol deeply unless sir asks exact format, but understand the meaning:

- 30 = start of sequence
 - 02 = integer marker
 - lengths tell how many bytes each number takes
 - then actual r and s bytes
 - then 1 extra byte called **sighash byte**
-

What is sighash byte?

This byte tells Bitcoin:

which parts of the transaction were hashed and signed

Example from lecture:

- 0x01 means all inputs and all outputs are signed
- 0x02 means all inputs signed but no output signed

So sighash byte is like a rule that tells the network how broad the signature commitment is.

Part 6: Why leading zeros matter

This is where the malleability problem starts.

Your lecture shows that the same signature can sometimes be encoded in slightly different valid ways. For example, one version may add a leading zero byte. That changes the byte length even though the actual signature meaning is still valid.

So:

- mathematically same spending intent
- slightly different byte representation
- transaction bytes change
- transaction hash changes

That is the doorway to transaction malleability.

Part 7: Transaction malleability

This is one of the biggest Week 6 topics.

What is txid?

txid = transaction ID

It is the hash-based ID of a transaction.

Wallets use it to track transactions.

What is transaction malleability?

Easy definition:

The transaction ID can be changed without changing the real effect of the transaction.

That means:

- same inputs spent
- same outputs created
- same money movement

but a different txid appears.

Why did this happen in legacy Bitcoin?

Because in legacy Bitcoin, the txid was calculated from the serialized transaction, and that serialization included **scriptSig**, which included the signature bytes.

If someone changed the signature's byte encoding in a still-valid way, then:

- serialized transaction changes
- hash changes
- txid changes

even though the payment meaning remains the same.

Why is malleability bad?

Your lecture gives a few reasons.

1. Unreliable txids

Wallet expects one txid, but a different version gets mined.

2. Wallet confusion

Wallet may think:

“my transaction disappeared”

even though equivalent version got confirmed.

3. Rebroadcast/modified version issue

Someone can take your pending transaction, tweak the signature encoding, and rebroadcast it.

4. Layer 2 protocols break

This is huge.

Off-chain systems want to create later transactions that refer to earlier txids.

But if earlier txid can change, those dependent transactions break.

That is why malleability was such a serious problem.

Part 8: SegWit

SegWit = **Segregated Witness**

What is the core idea?

Move signature/witness data outside the part used to compute txid.

That is the exam line.

In legacy Bitcoin, the proof data was inside scriptSig, and scriptSig affected txid.

In SegWit, witness data is separated. So txid becomes more stable.

What does “witness” mean?

Witness = the data that proves you had the right to spend.

Usually:

- signature
- public key
- or other spending proof

So witness is basically the authorization evidence.

What changes structurally in SegWit?

In SegWit transaction:

- scriptSig is empty or mostly empty
- unlocking data goes in a separate **witness** section
- marker and flag are added to indicate SegWit format

So the body changes.

Legacy vs SegWit in one simple line

Legacy:

- unlocking data sits in input's scriptSig

SegWit:

- unlocking data sits in separate witness field

That is the main structural difference.

Part 9: P2WPKH

P2WPKH = **Pay to Witness Public Key Hash**

This is basically the SegWit version of P2PKH.

Old P2PKH said:

“Spendable by whoever proves ownership of this public key hash.”

P2WPKH says the same core thing, but with witness-based structure.

What does the output look like?

Instead of showing the old visible opcode-heavy script, the output contains a compact witness program like:

- version 0
- length
- 20-byte pubkey hash

So it looks simpler on-chain.

Where are signature and public key now?

Not in scriptSig.

They are in the witness field.

Part 10: “No opcodes!” then how does validation still work?

This is such an important confusion point, and your lecture directly addresses it.

When you look at P2WPKH output, you do not see the old classic script:

- OP_DUP
- OP_HASH160
- ...
- OP_CHECKSIG

So it feels like: wait, where did all the checking go?

Answer:

Bitcoin internally expands the SegWit output into a virtual P2PKH-like script for validation.

So even though the on-chain output looks compact, internally validation still behaves like:

- duplicate pubkey
- hash pubkey
- compare hash
- verify signature

Meaning:

SegWit changed the format, not the basic ownership logic.

That is a very quiz-worthy point.

Part 11: P2WPKH execution story

Now let's simplify the stack execution flow.

Your lecture walks many slides through it. We can shrink it into the real story.

Step 1: ScriptSig is empty

Unlike legacy, unlocking data is not there.

Step 2: Bitcoin sees SegWit output

It expands the previous output into a virtual checking script.

Step 3: Witness pushes items

Witness provides:

- signature
- public key

These are placed on the stack.

Step 4: Script runs

The system checks:

- does this public key hash to the expected value?
- does the signature verify?

Step 5: Final result becomes True

If yes, transaction is valid.

So again, SegWit did not remove security checks. It only moved where the proof data lives.

Part 12: Multisig

Now Lecture 12. Multisig is a major topic.

What is multisig?

Multisig = **multiple signatures**

Easy meaning:

Instead of one key being enough, the transaction requires multiple approvals.

Example:

2-of-3 multisig

means:

- there are 3 allowed public keys
 - any 2 valid signatures are enough to spend
-

What does M-of-N mean?

- N = total authorized keys
- M = number of signatures required

So:

- 2-of-3 means 2 signatures needed out of 3 possible keys
 - 3-of-5 means 3 signatures needed out of 5
-

Why use multisig?

Your lecture gives these reasons:

Security

One lost/stolen key is not enough to lose funds.

Shared accounts

Company treasury, family wallet, etc.

Escrow

Buyer, seller, arbitrator in 2-of-3.

Lightning/payment channels

Often use 2-of-2.

So multisig is about shared control.

Part 13: How multisig works

Lecture says three phases.

1. Locking

Output is locked to a script like:

```
OP_2 <PubKey1> <PubKey2> <PubKey3> OP_3 OP_CHECKMULTISIG
```

This means:

“To spend me, you must provide 2 valid signatures from these 3 keys.”

2. Unlocking

Spender provides:

- required signatures
- the script itself or witness script proving the condition

3. Verification

Bitcoin checks whether enough signatures match the allowed keys.

That is the whole idea.

Part 14: P2WSH

P2WSH = **Pay to Witness Script Hash**

This is SegWit style, but instead of locking to a pubkey hash, it locks to the hash of a whole script.

That whole script can describe complex rules like multisig.

So:

- output stores only hash of witness script
- real witness script is revealed later when spending

Why do this?

Because the actual script may be large, so storing only its hash is more compact.

Part 15: The famous OP_0 bug in multisig

Your lecture highlights this weird thing: a dummy **OP_0** is included before signatures.

Why?

Because **OP_CHECKMULTISIG** historically has a bug where it consumes one extra stack item.

So Bitcoin multisig spending includes a dummy zero just to satisfy that bug.

For quiz language:

A dummy OP_0 is included because of the historical CHECKMULTISIG off-by-one bug.

That is enough.

Part 16: Multisig execution in simple steps

Let me tell it like a story.

Suppose previous output is a P2WSH multisig output.

Step 1: Check witness script hash

Bitcoin computes $\text{SHA256}(\text{witnessScript})$ and checks whether it matches the 32-byte hash stored in the old output.

If it does not match, fail.

If it matches, continue.

This proves the revealed witness script is the real one committed earlier.

Step 2: Push witness items except last

The last witness item is the witness script itself.

Other items like:

- OP_0 dummy
- signature A
- signature C

are pushed onto stack.

Step 3: Execute witness script

Script says something like:

- 2 required
- 3 total pubkeys
- check multisig

Step 4: Bitcoin pops:

- $n = 3$
- 3 public keys
- $m = 2$
- 2 signatures

Step 5: Verify signatures

It checks whether enough signatures match the allowed pubkeys.

If 2 valid signatures are found, success.

Step 6: Pop dummy 0

Because of bug.

Step 7: Push True

Transaction validates.

That is the conceptual flow.

Part 17: Block header

Now Lecture 13 moves to blocks.

What is a block header?

It is an 80-byte summary of the block.

It has 6 fields:

1. Version
2. Previous block hash
3. Merkle root hash
4. Timestamp
5. Target/difficulty
6. Nonce

Easy meanings

1. Version

Which protocol rules/signals this block uses.

2. Previous block hash

Links this block to parent block.

3. Merkle root

One compact fingerprint representing all transactions in block.

4. Timestamp

Approximate block creation time.

5. Target/difficulty

Defines how hard mining is.

6. Nonce

The value miners keep changing while trying hashes.

Block header hash

Bitcoin computes:

SHA256(SHA256(block header))

This double hash is the block header hash.

That is the thing miners try to make valid under the difficulty target.

Part 18: Why “coffee with Bitcoin” is a problem

Lecture 13 asks: can we buy coffee with Bitcoin?

This is a motivation topic.

Problems

1. 10-minute block time

Too slow for coffee.

2. Public visibility

Do we want every coffee purchase on public blockchain?

3. Throughput

Bitcoin base layer handles very few transactions per second.

4. Fee problem

Small payments can get hit by fees that are too large relative to amount.

So Bitcoin base layer is secure, but not ideal by itself for tiny everyday micropayments.

Part 19: Bitcoin Script can do more than one-key payments

Lecture 13 reminds you that Bitcoin Script can do many things.

It can:

- verify signatures with `OP_CHECKSIG`
- use conditions with `OP_IF`, `OP_ELSE`
- require multiple approvals with `OP_CHECKMULTISIG`
- use time locks with `OP_CHECKLOCKTIMEVERIFY`
- use delay locks with `OP_CHECKSEQUENCEVERIFY`

This matters because Bitcoin is not just “send money.”
It can encode rules.

Part 20: Miniscript

This is mostly conceptual.

Your lecture says:

Miniscript is like TypeScript for Bitcoin Script

Easy meaning:

- raw Bitcoin Script is low-level and hard to read
- Miniscript is more structured and human-friendly
- then it can be converted into actual Bitcoin Script

So it helps developers describe policies more safely.

Part 21: UTXO model revisited

Lecture 13 comes back to UTXO but in a practical validation way.

Important rules

Each output has:

- value
- scriptPubKey

Each input must reference:

- exactly one previous output

In legacy:

- input has scriptSig

In SegWit:

- scriptSig empty
- witness holds unlocking proof

Sum of inputs must be at least sum of outputs

If outputs are less than inputs, leftover becomes fee.

Part 22: Parent and child transaction

This is another likely quiz concept.

- **Parent transaction** = earlier transaction that created the output
- **Child transaction** = later transaction that spends it

So the child points to the parent's output and tries to unlock it.

Very clean sentence:

Parent creates the coin, child spends the coin.

Part 23: Transaction chaining

Transactions also form a chain.

That means:

- one transaction creates outputs
- later transaction spends them
- then creates new outputs
- later another transaction spends those

So not only blocks form a chain. Transactions also form a spending chain.

That is a nice conceptual point.

Part 24: UTXO database

This is very important.

UTXOs are scattered throughout blocks, but nodes keep a separate current database of all unspent outputs. In Bitcoin Core this is called **chainstate.db**.

Why maintain this?

To speed up validation.

Otherwise, every time a new transaction arrives, the node would have to search the whole blockchain history.

That would be too slow.

What does the UTXO database store?

Only the outputs that are still unspent.

Not everything historically.

Only the currently spendable coins.

Simplified validation algorithm

Lecture gives this in a very nice form.

For each new transaction:

1. Check whether referenced parent UTXOs exist in database
2. Check whether provided signature/unlocking proof matches owner/spending rule
3. Check input total is not less than output total
4. If valid:
 - remove spent parent UTXOs
 - add new child outputs as fresh UTXOs

This is exactly how Bitcoin's current spendable state moves forward.

Part 25: Change output

Even though Lecture 14 is excluded, Week 7 notes you uploaded also summarize the change-output idea and it connects directly with Lecture 13's UTXO logic, so I'll explain the concept only because it is fundamental and appears in your review notes.

Suppose Alice owns one UTXO worth 1.5 BTC and wants to pay Bob 0.5 BTC.

If she creates only one output of 0.5 BTC to Bob, then the leftover 1 BTC becomes fee.

That is valid, but stupid.

So she usually creates:

- 0.5 BTC to Bob
- 1 BTC back to herself as **change**

This is just like paying with a bigger currency note and receiving change back.

Very important for understanding normal Bitcoin transaction shape.

Part 26: Why transaction fee hurts micropayments

The lecture points out something very important:

Bitcoin fee depends mainly on transaction size in bytes, not on transaction value.

This means:

- a \$1 coffee payment
- and a \$100,000 payment

can pay similar fees if transaction sizes are similar.

That is why on-chain Bitcoin is bad for micropayments.

Small payment does not automatically mean small fee.

Part 27: Funding transaction and commitment transactions

Now the lecture introduces how to reduce fees using off-chain updates.

Idea

Instead of putting every tiny coffee payment on-chain:

1. Open a channel with one on-chain **funding transaction**
2. Make many small balance updates off-chain
3. Finally settle latest balance on-chain

So you pay fees mainly for:

- opening
- closing

not for every single coffee.

What is funding transaction?

It is the initial on-chain transaction that locks funds into a shared setup, often **2-of-2 multisig**.

That money becomes the money inside the channel.

What is a commitment transaction?

A commitment transaction is an off-chain transaction that represents the current balance split between the two channel parties.

Example:

Funding starts with 100,000 sats from Alice.

Initial commitment might say:

- Alice 100,000
- Bob 0

After Alice buys coffee for 2,000 sats:

new commitment says:

- Alice 98,000
- Bob 2,000

After another coffee update:

- Alice 95,000
- Bob 5,000

Only latest one matters.

Part 28: Why SegWit matters for payment channels

This is a very smart connection your lecture makes.

Legacy transaction format is bad for payment channels because txids could be malleated.

If txid changes, child transactions depending on that txid break.

SegWit solves this because witness/signatures are not part of txid calculation the same way.

So SegWit made secure payment-channel style constructions much more practical.

This is one of the most important conceptual links across the lectures:

SegWit was not just a formatting upgrade. It was crucial for advanced second-layer systems.

Quick memory sheet for the hardest terms

scriptPubKey

Locking script on output.

Defines spending condition.

scriptSig

Unlocking data in legacy input.

Usually signature + public key.

witness

Unlocking proof in SegWit.
Stored separately from main transaction body.

txid

Transaction ID.
Hash-based identifier of transaction.

transaction malleability

txid can change without changing real payment effect.

SegWit

Separates witness/signatures from txid-critical part.
Fixes malleability problem.

P2PKH

Pay to Public Key Hash.
Old style locking to hash of public key.

P2WPKH

SegWit version of pay to public key hash.

P2WSH

SegWit version of pay to witness script hash.
Used for larger/custom scripts like multisig.

multisig

Multiple signatures required.

M-of-N

M signatures needed out of N allowed keys.

OP_RETURN

Creates provably unspendable output, usually for data.

UTXO

Unspent Transaction Output.
Spendable coin piece.

parent transaction

Transaction that created an output.

child transaction

Transaction that spends that output.

change output

Extra output returning leftover value back to sender.

funding transaction

Initial on-chain transaction opening shared payment setup.

commitment transaction

Off-chain current-balance transaction inside payment channel.

Super likely quiz-style conceptual questions

1. What is the difference between scriptPubKey and scriptSig?

scriptPubKey is the locking script placed in an output. scriptSig is the unlocking data provided in a later input in legacy Bitcoin. Bitcoin executes the unlocking data against the locking condition to check whether the spender is authorized.

2. What is transaction malleability?

It is the problem where a transaction's txid can be changed without changing the actual inputs spent or outputs created, because legacy txid included signature bytes that could be re-encoded in another valid form.

3. How does SegWit fix malleability?

SegWit separates the witness data, such as signatures and public keys, from the transaction body used for txid calculation, so changing witness encoding does not change txid.

4. What is multisig?

A multisig transaction requires multiple signatures to authorize spending, such as 2-of-3 where any two of three authorized keys are enough.

5. Why is OP_0 used in multisig witness?

Because of a historical CHECKMULTISIG bug that consumes one extra stack item, so a dummy zero is added.

6. What is a UTXO?

A UTXO is an output from an earlier transaction that has not yet been spent and can be used as an input in a new transaction.

7. Why are change outputs needed?

Because Bitcoin spends old outputs whole. If an input is larger than the payment, the remaining value must be explicitly returned to the sender as a new output, otherwise it becomes fee.

8. Why is Bitcoin weak for micropayments on-chain?

Because block time is slow, throughput is limited, and transaction fees depend on transaction size rather than payment value.

Final super-short storyline of all 3 lectures

Lecture 11 says:

Legacy Bitcoin spends old outputs using scriptSig to satisfy scriptPubKey, but signatures inside scriptSig caused transaction malleability, so SegWit moved proof data into witness and made txids more stable.

Lecture 12 says:

Bitcoin can also lock outputs with more advanced rules like multisig, where M signatures out of N keys are required, often using SegWit script-hash structures like P2WSH.

Lecture 13 says:

Blocks are summarized by headers, Bitcoin uses the UTXO model and chainstate database for efficient validation, and because on-chain Bitcoin is slow/expensive for coffee-size payments, payment-channel ideas use funding and commitment transactions to move many micropayments off-chain.